A.U.R.A. — Systematic Literature Review

Air-Gapped Unified Responsive Agent: A Local, Multi-Agent, Security-Focused Computer-Use System

Muhammad Hunzala Hasan (25435) · Muhammad Taha Jabbar (25421) · Hafsa Ayub (25588) | Department of Computer Science, Iqra University | Final Year Project, Batch Fall-2026

1. Introduction

A.U.R.A. (Air-gapped Unified Responsive Agent) is a locally run, multi-agent computer-use system that converts natural-language commands into controlled operating-system actions without any cloud dependency. The project combines four research areas, local/edge LLM inference, multi-agent orchestration, OS-level automation with resilient UI interaction, and hardware-enforced agent safety, into a single deployable system for consumer hardware.

This Systematic Literature Review (SLR) surveys the peer-reviewed and technical literature underlying each architectural component of A.U.R.A., synthesizes it into design-relevant findings, positions the project against existing local-AI tooling and computer-use agents, and derives the system's functional and non-functional requirements directly from that synthesis. Consistent with standard SLR practice, this review is treated as a living technology-validation exercise where the literature and subsequent hands-on benchmarking indicated that an originally proposed component was not the strongest available choice, the design was revised accordingly and that revision is documented explicitly (Section 4.2) rather than retrofitted.

2. Research Questions

ID	Research Question
RQ1	What architectures exist for coordinating multiple LLM-driven agents on a single shared local reasoning model, and how do they compare to running separate models per agent?
RQ2	What is the current state of low-bit and quantized local LLM inference, and what are the practical trade-offs between a native 1.58-bit model and a standard 4-bit/8-bit quantized dense model for an agentic, tool-calling workload?
RQ3	How can operating-system automation be performed reliably on Windows without depending on cloud vision APIs, and how resilient is accessibility-tree-based control compared to screenshot/vision-based control?
RQ4	What fallback mechanisms exist for locating UI elements when accessibility metadata is unavailable or a third-party application's layout changes unexpectedly?
RQ5	What techniques exist for defending a tool-using LLM agent against indirect prompt injection and other untrusted-content attacks?
RQ6	What human-in-the-loop and hardware-based authorization mechanisms exist for constraining high-risk autonomous-agent actions?
RQ7	What memory architectures allow an LLM agent to retain user-specific context (preferences, file locations, habits) across sessions without requiring the model itself to memorize it?
RQ8	What on-device speech and biometric technologies are suitable for a resource-constrained IoT companion node (Raspberry Pi class hardware)?

3. Review Methodology

3.1 Search Strategy

Literature was identified primarily through arXiv (cs.CL, cs.AI, cs.CR, cs.HC categories), cross-checked where available against IEEE Xplore, ACM Digital Library, NeurIPS/ICLR/ACL proceedings, and official technical reports published directly by the originating research labs (Microsoft Research, Qwen Team, UC Berkeley). The search window was restricted to 2020–2026 to keep the review anchored to techniques that are practically deployable on current consumer hardware. Search terms were built by combining a component keyword (e.g., "multi-agent LLM", "1-bit quantization", "GUI grounding", "prompt injection", "human-in-the-loop agent", "on-device inference", "fingerprint presentation attack", "wake-word detection") with qualifiers such as "local", "edge", "agent", and "benchmark".

3.2 Inclusion / Exclusion Criteria

- Included: peer-reviewed papers, official technical reports from model developers, and recognized benchmarks published 2020–2026 that directly inform one of A.U.R.A.'s ten core features.
- Included: at least one foundational/earlier paper per theme where it establishes the technique later work builds on (e.g., the original BitNet and Toolformer papers).
- Excluded: non-technical blog posts, unverified marketing claims, and sources whose title/venue/year could not be independently confirmed against the live paper.
- Excluded: papers describing techniques requiring datacenter-scale hardware with no consumer-hardware relevance.

3.3 Study Selection

Each candidate source was verified directly (title, publication year, and link confirmed against the live paper or official model card) before inclusion. Sources were then organized into eleven thematic categories, each mapped to one or more of A.U.R.A.'s ten core features, as presented in Section 4. The initial selection (50 sources) was produced during the proposal stage, one additional source (Qwen3 Technical Report [51]) was added during this revision to ground the current main-model decision described in Section 4.2, bringing the total to 51 sources.

4. Thematic Literature Synthesis

4.1 Multi-Agent Orchestration (Feature 1)

AutoGen [1] and MetaGPT [2] establish that role-specialized agents coordinated through a shared conversational or procedural protocol can decompose complex tasks more reliably than a single monolithic agent, the architectural basis for A.U.R.A.'s Brain/Manager/Sub-Agent decomposition. ReAct [3] provides the reason-then-act loop underlying how a plan is turned into a sequence of tool calls, and Toolformer [6] underpins how an agent learns to select the correct tool/API for a given step. Generative Agents [4] demonstrates that persistent memory materially improves multi-step agent behaviour, motivating the memory layer discussed in Section 4.8. GAIA [5] and the 2024 multi-agent survey [7] give benchmark and landscape context for evaluating A.U.R.A.'s end-to-end task completion against the current state of the art.

Design implication: consistent with [1][2], A.U.R.A. does not require one separate loaded model per agent. A single shared reasoning model, given different prompts, tool sets, permissions, and memory scopes, can instantiate multiple logical agents, substantially reducing VRAM and RAM requirements on consumer hardware without abandoning the multi-agent design.

4.2 Hybrid / Low-Bit Local Reasoning Engine — Design Revision (Feature 2)

The original design (as reflected in the initial proposal) followed the 1-bit LLM literature [8][9][10] directly: BitNet b1.58 2B4T [9], the first natively-trained, publicly available 1.58-bit model, built on the ternary-weight foundation of

[8] and the earlier BitNet architecture [10], was proposed as a fast, low-latency “Brain/Manager” routing layer ahead of a separately loaded 7–8B quantized reasoning model. This was a reasonable literature-driven starting point: [8][9] show that ternary {-1, 0, 1} weights replace floating-point multiplication with addition, giving genuine CPU-side efficiency gains.

However, closer technical evaluation against the project's actual constraints surfaced two limitations not visible from the papers alone: (a) BitNet b1.58 2B4T's published context window is only 4,096 tokens, tight for an orchestrator that must hold a system prompt, tool schemas, conversation history, and retrieved memory simultaneously, and (b) realizing BitNet's efficiency claims in practice requires Microsoft's dedicated bitnet.cpp inference runtime rather than the standard llama.cpp/Transformers stack used for the rest of the pipeline, adding a second, less mature toolchain to maintain. Running a 2-billion-parameter model as the sole intent-classification/orchestration layer also concentrates a disproportionate amount of the system's language-understanding burden on the least capable model in the pipeline.

Revised decision: A.U.R.A. now uses a single shared local reasoning model, Qwen3.5-4B, from the Qwen3 model family [51], for both routing/classification and complex multi-step reasoning, invoked with different prompts and tool scopes per logical agent (Section 4.1). This removes the cross-runtime dependency, restores a much larger practical context window, and matches current evidence [16][17] that modern quantized dense models in the 4–9B range are competitive with far larger models on agentic, tool-calling workloads while remaining CPU/low-VRAM-inference-friendly. Standard post-training quantization methods, GPTQ [11], AWQ [12], LLM.int8() [13], and SmoothQuant [14], together with the systems-level serving survey [15], remain directly applicable to compressing this single model onto the target hardware. BitNet [8][9][10] is retained in this review as background on 1-bit architectures and as a documented, deliberately-not-adopted alternative for the fast-routing role, not as A.U.R.A.'s active technology stack.

4.3 On-Device / Edge LLM Inference (Feature 2)

The comprehensive review of on-device language models [16] lays out the privacy and latency case for local-only inference that directly motivates A.U.R.A.'s 100%-offline design. [17] provides evidence that CPU-only consumer laptops (no dedicated GPU) can run small/quantized LLMs effectively, which is relevant both to the project's minimum-hardware tier and to the Raspberry Pi companion node's local wake-word/lightweight-inference budget.

4.4 OS Automation via Accessibility Hooks (Feature 3)

OSWorld [18] provides the key empirical result underlying A.U.R.A.'s accessibility-first design, accessibility-tree-based control is faster and more reliable than pure vision-based screenshot analysis for the majority of desktop tasks. UFO [19] and its successor UFO2 [20] are the closest directly comparable Windows-automation agents (and are cited as market competitors in Section 5) and provide a benchmarking reference point for A.U.R.A.'s background-automation approach. Windows Agent Arena [21] offers a large-scale evaluation harness, and OS-ATLAS [22] contributes a generalist action-model formulation relevant to designing the sub-agent’s action spaces.

4.5 Vision Fallback for UI Resilience (Feature 4)

A cluster of GUI-grounding vision-language models, CogAgent [23], SeeClick [24], ScreenAI [25], ShowUI [26], Ferret-UI [27], UI-TARS [28], and the earlier foundational Pix2Struct [29], collectively demonstrate that a UI element can be relocated from a screenshot alone, without accessibility metadata, which is exactly the fallback scenario A.U.R.A. requires when a third-party application (e.g., WhatsApp Web) changes its layout. ScreenAgent [30] and the universal-visual-grounding work [31] show this can be done end-to-end and generalizes across applications, GUI-WORLD [32] supplies a dataset suitable for evaluating or fine-tuning the fallback module. Because current-generation reasoning models (including the Qwen3 family adopted in Section 4.2) increasingly ship with native multimodal capability, this fallback should first be tested against the main reasoning model's own vision capability before an additional dedicated vision model is added to the pipeline.

4.6 Zero-Trust Security & Prompt-Injection Defense (Feature 6)

BIPIA [33] and InjecAgent [34] provide the benchmark evidence that indirect prompt injection, malicious instructions embedded in web or document content, is a practical, demonstrated attack against tool-using LLM agents, directly justifying A.U.R.A.'s sanitize-before-reasoning design (stripping HTML/JavaScript so only plain text reaches the model). Progent [35] offers a privilege-control framework conceptually aligned with A.U.R.A.'s restricted-permission sandbox, and [36] documents one of the earliest instruction-override attack techniques that motivates treating all externally retrieved content as untrusted by default.

4.7 Hardware-in-the-Loop Security & Agent Safety (Features 7 & 8)

Magentic-UI [37] is the closest software analogue of A.U.R.A.'s human-in-the-loop design, implementing action guards for computer-use agents that pause for human confirmation. The International AI Safety Report 2026 [38] independently identifies human-in-the-loop oversight and sandboxing as key mitigations for autonomous-agent risk at a policy level, supporting the broader case for A.U.R.A.'s fingerprint-gated authorization rather than a software-only confirmation dialog.

4.8 Hierarchical Long-Term Memory & Retrieval (Feature 5)

MemGPT [39] introduces virtual-memory-style context management for LLM agents and is the direct architectural basis for A.U.R.A.'s ChromaDB-backed long-term memory. The RAG survey [40] provides the broader retrieval-augmented-generation foundation, rather than requiring the reasoning model to memorize user-specific facts, relevant context (file paths, preferences, prior task outcomes) is retrieved and injected at inference time, which, per the revised architecture in Section 4.2, also reduces the intelligence burden placed on the smaller Qwen3.5-4B model.

4.9 Voice Input: Wake-Word Detection & Speech Recognition (IoT Node)

Whisper [41] is the proposed speech-to-text model for converting spoken commands to text on the IoT node. Howl [42], Sparse Binarization [43], and the Keyword Transformer [44] provide progressively lighter-weight wake-word/keyword-spotting architectures suited to the Raspberry Pi's constrained compute budget, allowing the companion node to listen continuously without materially affecting the main reasoning pipeline's resources.

4.10 Biometric Fingerprint Authentication (Feature 8)

AFR-Net [45] represents a modern deep-learning approach to fingerprint matching relevant to the AS608 sensor's verification pipeline. The presentation-attack-detection survey [46] and the hybrid CNN+LSTM authentication approach [47] are directly relevant to hardening the fingerprint gate against spoofing and to setting an accuracy target for false-acceptance/false-rejection rates on the HITL authorization step.

4.11 IoT Node Security & Encrypted Communication (Features 9 & 10)

The edge/IoT security survey [48] frames the general threat surface of the Raspberry Pi companion node. The lightweight session-key rekeying framework [49], tested on an ESP32–Raspberry Pi 5 testbed nearly identical to A.U.R.A.'s own hardware, is directly applicable to securing the PC–Pi WebSocket link used for the fingerprint-approval handshake. The lightweight-cryptography survey [50] informs the choice of encryption primitives appropriate for a resource-constrained device.

5. Comparative Analysis of Existing Systems

A.U.R.A.'s literature-grounded contribution is not any single technique in isolation, local inference, accessibility automation, and human-in-the-loop gating each have independent prior art, but their integration into one air-gapped, hardware-authorized system. The table below positions A.U.R.A. against the closest existing systems identified in the market/prior-art review.

System	Approach	Gap vs. A.U.R.A.
Claude Computer Use	Captures UI screenshots, sends to a cloud API for reasoning [Anthropic docs].	Cloud-dependent, screen data leaves the device, network latency.
Microsoft UFO / UFO2 [19][20]	Automates Windows UI using accessibility hooks plus paid GPT-4V API calls.	Ongoing per-token API cost, not air-gapped, no hardware authorization layer.
Open Interpreter	Terminal-based agent executing Python code locally.	No physical safeguard against a destructive or malicious action.
Ollama / LM Studio / Jan (local runtimes)	General-purpose local model runners, not agentic or OS-integrated by themselves.	Provide inference only, no orchestration, OS automation, or security layer.
GPT-6 Astra (OpenAI)	OpenAI's flagship computer-use/agentic model (released September 2026), cloud-hosted via ChatGPT, the OpenAI API, Microsoft Azure, and AWS Bedrock. State-of-the-art on computer-use and agentic benchmarks (e.g., OSWorld, ScreenSpot-Pro), with a 1.05M-token context window and native multi-step tool orchestration.	Cloud-dependent by design, all reasoning and screen/task data are processed on OpenAI's infrastructure, with no offline or on-device mode, requires an active subscription/API billing, has no hardware-enforced (biometric) authorization layer for high-risk actions. A.U.R.A. targets the same end-to-end computer-use experience but fully air-gapped, at zero ongoing cost, with fingerprint-gated HITL security.

6. Research Gap Summary

- No reviewed system combines fully local (air-gapped) LLM reasoning with hardware-enforced (fingerprint) authorization for high-risk OS actions, existing HITL work [37][38] is software-confirmation only.
- Accessibility-first automation with a vision-language fallback [18][23–32] is well established individually, but not commonly paired with a zero-trust, sanitize-before-reasoning content pipeline [33–36] in one shipped local agent.
- Published local-AI ecosystems [16][17] and runtimes (Ollama, LM Studio, Jan, Open Interpreter) provide inference or single-agent automation, not an integrated multi-agent orchestration layer sharing one model [1][2][6] under a deterministic security/permission boundary.
- The 1-bit vs. standard-quantization trade-off for an agentic routing role (Section 4.2) is under-documented in practice-oriented terms, this SLR's finding, that context-window and toolchain maturity can outweigh raw bit-width efficiency for a small agentic model, is itself a design-relevant contribution of this review.

7. Functional Requirements

Derived directly from the feature list and the literature synthesis in Section 4.

ID	Requirement
FR-1	The system shall accept natural-language commands via typed text or voice input.
FR-2	The system shall interpret user intent and decompose a multi-step request into an executable plan.
FR-3	The system shall route each plan step to the appropriate logical agent (Browser, File/OS, Communication, Research, Security) sharing the single reasoning model.
FR-4	The system shall perform OS-level automation via accessibility APIs, targeting specific processes/UI elements without forcing target windows to the foreground.
FR-5	The system shall fall back to a vision-based method to relocate a UI element when accessibility-tree lookup fails.
FR-6	The system shall store and retrieve user-specific long-term context (preferences, file paths, prior task outcomes) via a local vector memory store.
FR-7	The system shall strip HTML/JavaScript from externally retrieved web or document content before it reaches the reasoning model.
FR-8	The system shall classify each requested action by risk level using a deterministic policy engine external to the reasoning model.
FR-9	The system shall freeze execution of any high-risk action until the user verifies identity via the fingerprint sensor on the IoT companion node.
FR-10	The system shall capture voice commands and detect a wake-word via the Raspberry Pi companion node, independent of the primary device's display or lid state.
FR-11	The system shall provide silent visual feedback of task status via the companion node's projector output.
FR-12	The system shall log key decisions, tool calls, and authorization outcomes for later audit.
FR-13	The system shall apply randomized human-like interaction timing and per-platform rate limits when automating third-party web platforms.
FR-14	The system shall encrypt all locally stored memory and state data using AES-256.
FR-15	The system shall verify the outcome of an executed action before reporting task completion to the user.

8. Non-Functional Requirements

ID	Category	Requirement
NFR-1	**Privacy**	The system shall not transmit task data, screenshots, or user context to any external network or cloud service; all inference shall occur on-device.
NFR-2	**Performance**	The primary reasoning response shall complete within an interactive latency bound on the reference hardware (≥ 6GB VRAM, 16GB RAM), to be established via A.U.R.A.-specific benchmarking.
NFR-3	**Portability**	The system shall run on a consumer-grade Windows 11 laptop without requiring a dedicated server or cloud subscription.
NFR-4	**Adaptability**	The system shall select an appropriate model/quantization configuration based on detected hardware (VRAM tier) without requiring manual user configuration.
NFR-5	**Security**	The reasoning model shall never hold unmediated authority over OS execution, every action shall pass through a deterministic policy and permission layer before execution.
NFR-6	**Reliability**	The system shall maintain a defined maximum tool-call/task failure rate across a representative benchmark task set (Section 3).
NFR-7	**Resource Efficiency**	The system's peak VRAM footprint on the reference hardware shall remain within the available budget, accounting for concurrent OS and application usage.
NFR-8	**Maintainability**	The multi-agent design shall allow a new logical agent (prompt, tool set, permission scope) to be added without loading an additional model instance.
NFR-9	**Auditability**	All high-risk action attempts, approvals, and denials shall be recorded in a local, tamper-evident log.
NFR-10	**Accessibility**	The system shall support hands-free, headless operation for users who cannot use the primary display or physically operate the laptop.

9. Conclusion

The reviewed literature supports each of A.U.R.A.'s core features individuall, multi-agent orchestration, local quantized inference, accessibility-first automation with vision fallback, prompt-injection-resistant sandboxing, human-in-the-loop hardware authorization, and persistent local memory. No single reviewed system integrates all of these under one air-gapped, biometrically-gated architecture, which constitutes A.U.R.A.'s research contribution. This review also produced one concrete design revision, the original BitNet-based fast-routing layer was replaced with a single shared Qwen3-family reasoning model (Section 4.2) after technical evaluation showed the context-window and toolchain costs of the 1-bit approach outweighed its efficiency benefit for this specific agentic workload, an outcome consistent with the project's stated principle that the implementation technology stack, unlike the core idea, remains open to revision as it is validated against real hardware and real tasks.

References

[1] Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155.

[2] Hong, S. et al. (2023). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. arXiv:2308.00352.

[3] Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.

[4] Park, J.S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.

[5] Mialon, G. et al. (2023). GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983.

[6] Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

[7] (2024). A Survey on LLM-based Multi-Agent System: Recent Advances and New Frontiers in Application. arXiv:2412.17481.

[8] Ma, S. et al. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764.

[9] Ma, S. et al. (2025). BitNet b1.58 2B4T Technical Report. arXiv:2504.12285.

[10] Wang, H. et al. (2023). BitNet: Scaling 1-bit Transformers for Large Language Models. arXiv:2310.11453.

[11] Frantar, E. et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.

[12] Lin, J. et al. (2023). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978.

[13] Dettmers, T. et al. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339.

[14] Xiao, G. et al. (2022). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438.

[15] (2023). Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. arXiv:2312.15234.

[16] (2024). On-Device Language Models: A Comprehensive Review. arXiv:2409.00088.

[17] (2025). Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference. arXiv:2505.06461.

[18] Xie, T. et al. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972.

[19] Zhang, C. et al. (2024). UFO: A UI-Focused Agent for Windows OS Interaction. arXiv:2402.07939.

[20] Microsoft Research (2025). UFO2: The Desktop AgentOS. arXiv:2504.14603.

[21] (2024). Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. arXiv:2409.08264.

[22] (2024). OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. arXiv:2410.23218.

[23] (2023). CogAgent: A Visual Language Model for GUI Agents. arXiv:2312.08914.

[24] (2024). SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. arXiv:2401.10935.

[25] (2024). ScreenAI: A Vision-Language Model for UI and Infographics Understanding. arXiv:2402.04615.

[26] (2024). ShowUI: One Vision-Language-Action Model for GUI Visual Agent. arXiv:2411.17465.

[27] (2024). Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. arXiv:2404.05719.

[28] (2025). UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326.

[29] (2022). Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. arXiv:2210.03347.

[30] (2024). ScreenAgent: A Vision Language Model-driven Computer Control Agent. arXiv:2402.07945.

[31] (2024). Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. arXiv:2410.05243.

[32] (2024). GUI-WORLD: A Dataset for GUI-oriented Multimodal LLM-based Agents. arXiv:2406.10819.

[33] Yi, J. et al. (2023). Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. arXiv:2312.14197.

[34] Zhan, Q. et al. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv:2403.02691.

[35] Shi, T. et al. (2025). Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703.

[36] Perez, F. & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques for Language Models. arXiv:2211.09527.

[37] Microsoft Research (2025). Magentic-UI: Towards Human-in-the-loop Agentic Systems. arXiv:2507.22358.

[38] (2026). International AI Safety Report 2026. arXiv:2602.21012.

[39] Packer, C. et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

[40] Gao, Y. et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[41] Radford, A. et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). arXiv:2212.04356.

[42] (2020). Howl: A Deployed, Open-Source Wake Word Detection System. arXiv:2008.09606.

[43] (2024). Sparse Binarization for Fast Keyword Spotting. arXiv:2406.06634.

[44] (2021). Keyword Transformer: A Self-Attention Model for Keyword Spotting. arXiv:2104.00769.

[45] (2022). AFR-Net: Attention-Driven Fingerprint Recognition Network. arXiv:2211.13897.

[46] (2023). Deep Learning based Fingerprint Presentation Attack Detection: A Comprehensive Survey. arXiv:2305.17522.

[47] Hussian, A. et al. (2025). A Hybrid Deep Learning Approach for Secure Biometric Authentication Using Fingerprint Data. Computers, 14(5), 178.

[48] (2020). A Survey on Security and Privacy Issues in Edge Computing-Assisted Internet of Things. arXiv:2008.03252.

[49] Rakshit, H. et al. (2025). Lightweight Session-Key Rekeying Framework for Secure IoT-Edge Communication. arXiv:2511.02924.

[50] (2020). Current Lightweight Cryptography Protocols in Smart City IoT Networks: A Survey. arXiv:2010.00852.

[51] Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. (Foundational report for the Qwen3 model family; Qwen3.5-4B, A.U.R.A.'s current main reasoning model, is a subsequent release within this lineage.)